

[39] Fast approximate nearest neighbor search with the navigating spreading-out graph

저자: C. Fu, C. Xiang, C. Wang, and D. Cai **학술지:** Proc. VLDB Endow., vol. 12, no. 5, p. 461-474, 2019 **주제:** NSG(Navigating Spreading-out Graph) - 그래프 기반 ANN 인덱싱

요약

본 논문은 HNSW와 경쟁하는 또 다른 강력한 그래프 기반 ANN 인덱싱 기법인 NSG(Navigating Spreading-out Graph, 네비게이팅 스프레딩아웃 그래프)를 제시한다. NSG는 단일 계층의 그래프 구조를 사용하면서도 HNSW와 유사한 검색 성능을 달성하고, 메모리 효율성과 구성 속도 면에서 우위를 가진다.

NSG의 핵심은 "spreading-out" 개념으로, 각 노드가 다양한 방향의 이웃들을 균형 있게 연결하도록 한다. 이는 그래프의 네비게이팅 능력(navigability)을 높이면서도 불필요한 간선을 줄인다.

본 논문과의 관계: Exqutor가 HNSW 대신 NSG를 사용할 경우의 성능 특성을 이해하는 데 중요하다. 단일 계층 구조가 필터링과 결합될 때 메모리 효율성과 검색 정확도 간의 트레이드오프를 분석할 수 있다.

상세분석

39.1 HNSW와 NSG의 비교

HNSW (Hierarchical): - 다중 계층 구조 (hierarchical) - 상단부터 하단으로 네비게이팅 - 삽입 시간 많이 소요 - 높은 메모리 사용량

NSG (Navigating Spreading-out): - 단일 계층 구조 (flat) - 직접 그래프 구성 - 더 빠른 구성 시간 - 더 낮은 메모리 오버헤드 - 검색 성능은 HNSW와 경쟁 가능

39.2 Spreading-out 개념

네비게이팅 능력(Navigability): - 임의의 두 점 사이의 최단 경로가 짧은 성질 - 검색의 "점프(hop)" 거리 최소화 - HNSW처럼 $O(\log n)$ 경로 길이 달성

Spreading-out 원리:

각 노드 u 에 대해:

1. u 와의 거리에 따라 모든 데이터 점을 정렬
2. 상위 M 개를 $candidates$ 로 선택
3. Spreading-out 휴리스틱 적용:
 - 점진적으로 $candidates$ 중 서로 먼 점들 선택
 - 다양한 방향 커버리지 확보
 - 제한된 수의 간선만 유지

목표: - 각 노드가 로컬 영역뿐 아니라 먼 영역도 커버 - 네트워크의 작은 세계 특성 보존 - 필요한 간선 수 최소화

39.3 NSG 구성 알고리즘

단계 1: 초기 그래프 생성

```
for each point p in dataset:
    // 모든 점과의 거리 계산
    distances = calculate_distances(p, all_points)

    // 상위 K개 근접 점 선택 ( $K \gg M$ )
    candidates = top_K_nearest(distances)

    // p의 이웃으로 설정
    neighbors[p] = candidates
```

단계 2: Spreading-out 휴리스틱

```
function spreading_out(candidates, M):
    result = []
    selected = [first_element(candidates)]

    while |result| < M:
        // 현재까지 선택된 점들로부터 가장 먼 점 선택
        farthest = argmax_distance_to_selected(candidates - selected)
        selected.add(farthest)
        result.append(farthest)

    return result
```

단계 3: 상호 연결(Reciprocal Links)

```
for each point p with neighbors N(p):
    for each neighbor q in N(p):
        // q에서 p로 역방향 링크 추가
        add_reverse_link(q, p)
```

단계 4: 반복적 개선 (그래프 최적화)

```
for iteration in 1 to num_iterations:
    for each point p:
        // 현재 이웃들
        current_neighbors = neighbors[p]

        // 이웃의 이웃들 탐색 (2-hop 친구)
        friend_candidates = union(neighbors[neighbor]
                                   for neighbor in current_neighbors)

        // Spreading-out으로 새로운 이웃 선택
        new_neighbors = spreading_out(friend_candidates, M)

        // 더 나으면 업데이트
        if quality(new_neighbors) > quality(current_neighbors):
            neighbors[p] = new_neighbors
```

39.4 검색 알고리즘

쿼리 처리:

```

procedure search_NSG(query_q, start_node, ef):
    visited = {start_node}
    candidates = {start_node}
    w = {start_node} // 결과 집합

    while not empty(candidates):
        // 현재 가장 가까운 점
        current = get_nearest_from_candidates(candidates)

        // 더 이상의 개선 불가능하면 종료
        if distance(current, q) > distance(farthest_in_w, q):
            break

        // current의 이웃 탐색
        for neighbor in neighbors[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                d = distance(neighbor, q)

                // 더 좋은 점을 찾았으면 추가
                if d < distance(farthest_in_w, q) or |w| < ef:
                    candidates.add(neighbor)
                    w.add(neighbor)

                // 결과 집합 유지
                if |w| > ef:
                    w.remove(farthest_point)

    return top_K_nearest_from_w(w, K)

```

39.5 시작점(Entry Point) 선택

NSG의 시작점 전략: - HNSW처럼 여러 계층의 진입점 불필요 - 데이터 중심(centroid) 점 사용 또는 무작위 시작
- 그래프 구조 최적화로 어디서 시작해도 비슷한 성능

시작점 선택 알고리즘:

```

entry_point = centroid of all points
또는
entry_point = random point from dataset

```

39.6 매개변수 분석

M (차수 제한): - 각 노드의 최대 이웃 수 - HNSW의 M과 유사 (보통 16~64) - M 증가: 검색 정확도 향상, 메모리 증가

K (초기 후보 크기): - Spreading-out 휴리스틱의 입력 크기 - 보통 M보다 훨씬 큼 ($K = 50M \sim 100M$) - K 증가: 더 나은 이웃 선택, 구성 시간 증가

ef (검색 확장 계수): - HNSW와 동일한 역할 - 검색 시간과 정확도 트레이드오프 제어

반복 횟수: - 그래프 최적화 반복 (보통 3~5회) - 각 반복마다 그래프 품질 향상

39.7 계산 복잡도 분석

구성 시간:

초기 그래프: $O(n * \log n)$ 거리 계산 + 정렬
정렬: $O(\log n)$ per point
최적화: $O(n * M * K)$ 반복 개선

전체: $O(n * K * \log n) \sim O(n * K * \text{상수})$

메모리 사용:

HNSW: 대략 64 바이트/점 ($M=16$)
NSG: 대략 32 바이트/점 ($M=16$)
→ NSG가 약 50% 메모리 절약

검색 시간:

평균: $O(\log n)$ 또는 $O(\sqrt{n})$
ef 파라미터에 따라 선형 계수 변동

39.8 NSG vs HNSW 성능 비교

지표	HNSW	NSG
구성 시간	중간~높음	낮음
메모리 사용	높음	낮음
검색 속도	매우 빠름	매우 빠름
검색 정확도	우수	우수
삽입/삭제	지원	어려움
구현 복잡도	중간	낮음

39.9 NSG의 장점

메모리 효율성: - 단일 계층으로 오버헤드 감소 - HNSW보다 약 50% 적은 메모리 - 대규모 데이터셋에 유리

구성 속도: - 다중 계층 유지 불필요 - 병렬 구성 용이 - 점진적 삽입보다 배치 구성 최적

구현 단순성: - HNSW보다 이해하고 구현하기 쉬움 - 페이지 참조 횟수 감소로 캐시 효율성 향상

수렴성: - 반복적 개선으로 점진적 성능 향상 - 구성 중단 가능성 (부분적 완성도 가능)

39.10 NSG의 단점

동적 작업: - 삽입/삭제가 비효율적 - 그래프 구조 재조정 필요 - HNSW가 더 적합

수렴 속도: - 반복 횟수에 따라 성능이 증가 - 충분한 반복 없으면 성능 저하

시작점 민감성: - 시작점 선택이 검색 성능에 영향 - 좋은 시작점 선택 휴리스틱 필요

39.11 필터링과 NSG의 통합

필터링 전략: - NSG 검색 중 필터 조건 적용 - 필터 조건을 만족하지 않는 노드는 스킵 - 필터 선택도에 따라 검색 범위 동적 조정

성능 특성: - 필터 선택도 높음: 넓은 검색 범위 필요 - 필터 선택도 낮음: 빠른 수렴 - 단일 계층 구조로 일관된 성능

추가 제기 문제

1. **NSG와 필터 인덱싱:** NSG의 단일 계층 구조가 다중 필터 조건과 결합될 때, 필터 조건별 별도의 그래프를 유지하는 것이 HNSW보다 메모리 효율적일까?
2. **시작점 선택 최적화:** 특정 필터 조건에 최적화된 시작점을 동적으로 선택할 수 있을까? 필터 조건별로 서로 다른 시작점을 유지하는 방식의 효과는?
3. **반복적 개선의 필터 적응:** NSG의 반복적 개선 단계에서 필터 조건을 고려하여 이웃을 선택하면 어떤 성능 향상을 기대할 수 있을까?
4. **온라인 학습:** 요청 패턴에 따라 NSG를 실시간으로 재구성할 수 있을까? 자주 함께 나타나는 필터 조건들에 최적화된 그래프 구조를 학습할 수 있을까?
5. **메모리 제약 환경에서의 NSG:** 메모리가 매우 제한된 환경(엣지 디바이스)에서 NSG와 필터링을 결합했을 때의 성능은?
6. **하이브리드 접근:** HNSW의 상위 계층과 NSG의 최적화된 단일 계층을 결합한 하이브리드 구조의 가능성은?

7. **필터 조건의 그래프 구조화**: 속성 기반 필터들의 유사도를 기반으로 필터 공간 내에서 NSG 그래프를 구성할 수 있을까?
8. **근사도 분석**: NSG가 HNSW보다 메모리 효율적이면서도 비슷한 정확도를 유지하는 이유를 필터링 관점에서 분석하면?
9. **확장성 한계**: NSG의 메모리 효율성에도 불구하고, 수십억 개 점 규모에서 필터링과 함께 사용할 때의 한계는?